

Lab 2.1

C Libraries, Linking, and ELF Executable Structure

Student: Binesh Shrestha

Environment: **Ubuntu Linux / WSL / VM and version**

Date: **date of submission**

Table of Contents

- Task 1: Process Information Program and C Library Files
- Task 2: Static and Dynamic Linking
- Task 3: ELF Executable Structure
- Knowledge Check Questions
- Conclusion

List of Figures

Figure 1: Development tools and version information

Figure 2: Source code of procinfo.c

Figure 3: Compilation and execution of procinfo

Figure 4: Header file locations and function declarations

Figure 5: Shared and static C library files

Figure 6: Size and file-type comparison of the dynamic and static executables

Figure 7: Dynamic section of the dynamic and static executables

Figure 8: Runtime dependencies resolved with ldd

Figure 9: ELF header of the dynamically linked executable

Figure 10: Program headers, interpreter path, LOAD segments and stack permissions

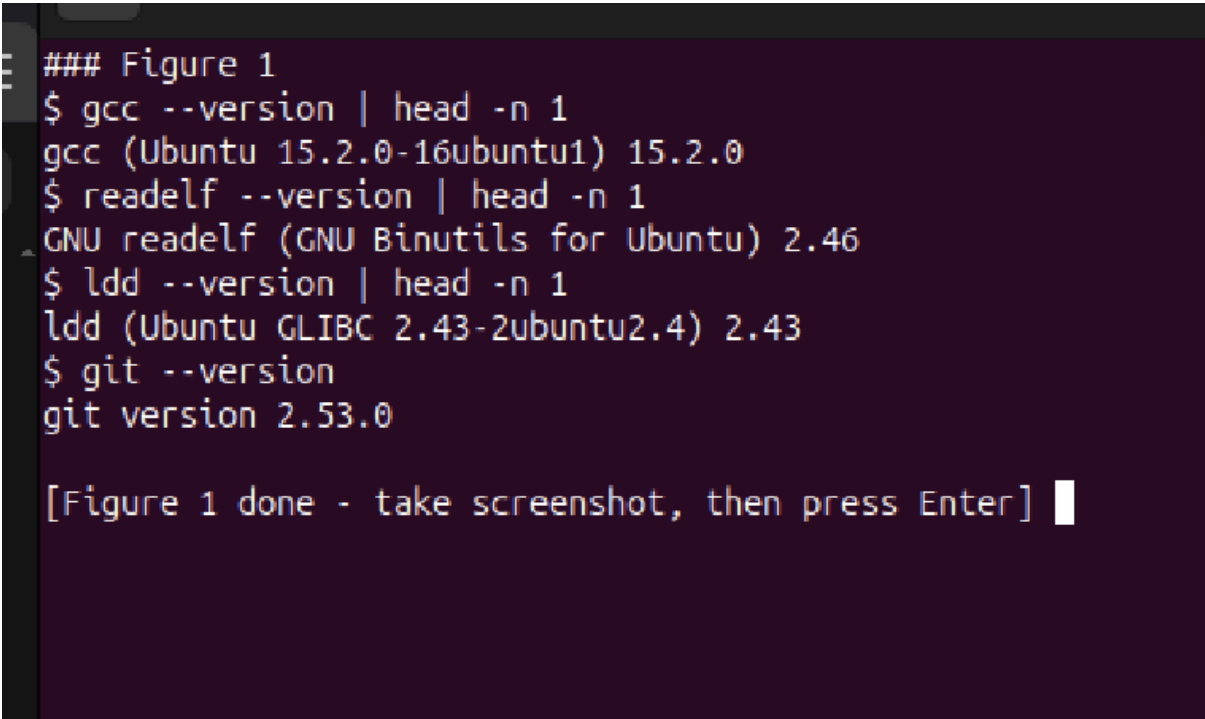
Task 1: Process Information Program and C Library Files

Objective

This task shows how a C program relies on two separate parts of the C Standard Library. Header files supply declarations to the compiler, and precompiled library files supply the machine code that actually runs. On Linux the library is glibc, and the process-information program in this lab uses printf, getpid, getppid, time, localtime and asctime from it.

Development Tools

Before starting, I checked that GCC, readelf, ldd and Git were installed and noted their versions. State whether all tools were already installed or whether you had to install any, e.g. build-essential.

A terminal window with a dark purple background and light-colored text. It shows the output of several commands to check the versions of development tools. The commands and their outputs are: gcc --version | head -n 1 (gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0), readelf --version | head -n 1 (GNU readelf (GNU Binutils for Ubuntu) 2.46), ldd --version | head -n 1 (ldd (Ubuntu GLIBC 2.43-2ubuntu2.4) 2.43), and git --version (git version 2.53.0). At the bottom, there is a prompt message: [Figure 1 done - take screenshot, then press Enter] followed by a cursor.

```
### Figure 1
$ gcc --version | head -n 1
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
$ readelf --version | head -n 1
GNU readelf (GNU Binutils for Ubuntu) 2.46
$ ldd --version | head -n 1
ldd (Ubuntu GLIBC 2.43-2ubuntu2.4) 2.43
$ git --version
git version 2.53.0

[Figure 1 done - take screenshot, then press Enter]
```

Figure 1: Development tools and version information

Source Code

The program was saved as procinfo.c in a Lab2_1 working directory. It reads the process ID, parent process ID and current local time, then prints them along with a fixed executable path string.

```

t 4 16:36
dada@dada: ~/Lab2_1
### Figure 2
$ cat procinfo.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>

int main(void) {
    // Get current process ID
    pid_t my_pid = getpid();
    pid_t my_ppid = getppid();

    // Get current time
    time_t current_time = time(NULL);
    struct tm *time_info = localtime(&current_time);

    // Format and display information
    printf("=== Process Information ===\n");
    printf("My Process ID (PID): %d\n", my_pid);
    printf("Parent Process ID: %d\n", my_ppid);
    printf("Current Time: %s", asctime(time_info));
    printf("Executable Path: /proc/self/exe\n");

    return 0;
}

[Figure 2 done - take screenshot, then press Enter]

```

Figure 2: Source code of `procinfo.c`

Compilation and Verification

The program was compiled with `-Wall -Wextra` so that GCC would report any warnings. Describe the result, e.g. no warnings or errors. Running it printed a PID of your PID, a parent PID of your PPID, the current time, and the executable path.

```

mkdir -p ~/Lab2_1 && cd ~/Lab2_1
gcc -Wall -Wextra procinfo.c -o procinfo
./procinfo

```

```

### Figure 3
$ gcc -Wall -Wextra procinfo.c -o procinfo
$ ./procinfo
=== Process Information ===
My Process ID (PID): 10513
Parent Process ID: 10143
Current Time: Sun Oct 4 16:50:18 2026
Executable Path: /proc/self/exe

[Figure 3 done - take screenshot, then press Enter]

```

Figure 3: Compilation and execution of procinfo

Header File Locations and Declarations

The standard headers live under `/usr/include`. Searching them showed that `stdio.h` declares `printf` and `unistd.h` declares `getpid`. These lines only describe the function signature, so the compiler knows how to call them. They contain no implementation, which means the compiler still has no idea where the real code is.

```

### Figure 4
$ ls -l /usr/include/stdio.h /usr/include/unistd.h
-rw-r--r-- 1 root root 34258 Sep  3 14:09 /usr/include/stdio.h
-rw-r--r-- 1 root root 45053 Sep  3 14:09 /usr/include/unistd.h
$ grep -n -m 1 "extern int printf" /usr/include/stdio.h
370:extern int printf (const char *__restrict __format, ...);
$ grep -n -m 1 "getpid (void)" /usr/include/unistd.h
650:extern __pid_t getpid (void) __THROW;

[Figure 4 done - take screenshot, then press Enter]

```

Figure 4: Header file locations and function declarations

C Standard Library Files

The compiled implementation is stored in library files. I located the shared library `libc.so.6`, which is loaded at runtime, and the static archive `libc.a`, which the linker can copy into an executable. On my system `libc.so.6` is `size` and `libc.a` is `size`.



```
dada@dada: ~/Lab2_1
### Figure 5
$ ls -lh /usr/lib/aarch64-linux-gnu/libc.so.6
-rwxr-xr-x 1 root root 1.8M Sep  3 14:09 /usr/lib/aarch64-linux-gnu/libc.so.6
$ ls -lh /usr/lib/aarch64-linux-gnu/libc.a
-rw-r--r-- 1 root root 5.5M Sep  3 14:09 /usr/lib/aarch64-linux-gnu/libc.a

[Figure 5 done - take screenshot, then press Enter]
```

Figure 5: Shared and static C library files

Task 1 Summary

Headers in `/usr/include` tell the compiler what a function looks like, while the compiled code sits separately in the system library directories. A program needs both to build and run.

Task 2: Static and Dynamic Linking

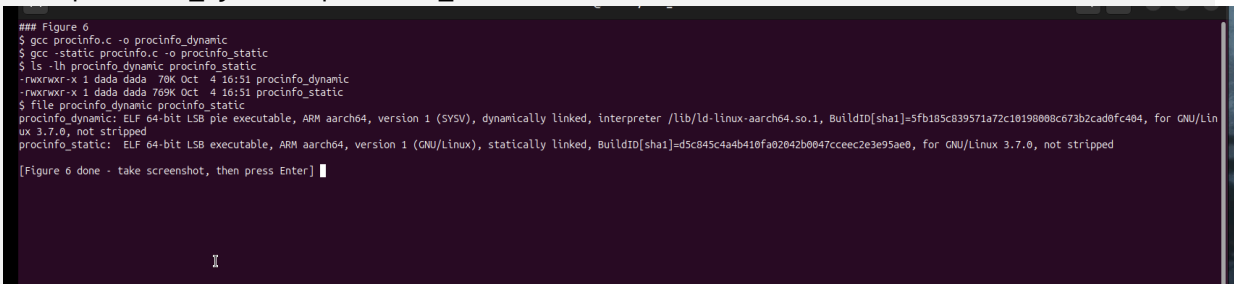
Theoretical Concept

Linking is the step that connects a function name such as `printf` to the actual code. In dynamic linking the executable only records which shared libraries it needs, and the dynamic linker loads them when the program starts. In static linking the linker copies the needed code out of the `.a` archive into the executable at build time. Dynamic executables are smaller and share library code in memory, while static executables are larger but have no runtime library dependencies.

Executable Comparison

The same source file was built both ways. The dynamically linked executable was **size** and the statically linked one was **size**. The `file` command reported the first as dynamically linked with an interpreter and the second as statically linked. The size gap exists because the static binary carries the library code it uses inside itself.

```
gcc procinfo.c -o procinfo_dynamic
gcc -static procinfo.c -o procinfo_static
ls -lh procinfo_dynamic procinfo_static
file procinfo_dynamic procinfo_static
```



```
### Figure 6
$ gcc procinfo.c -o procinfo_dynamic
$ gcc -static procinfo.c -o procinfo_static
$ ls -lh procinfo_dynamic procinfo_static
-rwxrwxr-x 1 dada dada 70K Oct  4 16:51 procinfo_dynamic
-rwxrwxr-x 1 dada dada 769K Oct  4 16:51 procinfo_static
$ file procinfo_dynamic procinfo_static
procinfo_dynamic: ELF 64-bit LSB pie executable, ARM aarch64, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux-aarch64.so.1, BuildID[shai]=5fb185c839571a72c10198008c673b2cad0fc404, for GNU/Linux 3.7.0, not stripped
procinfo_static: ELF 64-bit LSB executable, ARM aarch64, version 1 (GNU/Linux), statically linked, BuildID[shai]=d5c845c4a4b410fa02042b0047ccec2e3e95ae8, for GNU/Linux 3.7.0, not stripped
[Figure 6 done - take screenshot, then press Enter]
```

Figure 6: Size and file-type comparison of the dynamic and static executables

Dynamic Section

The dynamic executable has a `NEEDED` entry for `libc.so.6`, which tells the dynamic linker what to load. The static executable has no dynamic section at all, since nothing needs to be loaded at runtime.

```

dada@dada: ~/Lab2_1
### Figure 7
$ readelf -d procinfo_dynamic | grep NEEDED
0x0000000000000001 (NEEDED)      Shared library: [libc.so.6]
0x0000000000000001 (NEEDED)      Shared library: [ld-linux-aarch64.so.1]
$ readelf -d procinfo_static

There is no dynamic section in this file.

[Figure 7 done - take screenshot, then press Enter]

```

Figure 7: Dynamic section of the dynamic and static executables

Runtime Dependencies

Where readelf only lists library names, ldd resolves them to real paths. For the dynamic executable it showed linux-vdso.so.1, libc.so.6 at **resolved path**, and the dynamic linker ld-linux-x86-64.so.2. For the static executable it reported "not a dynamic executable".

```

dada@dada: ~/Lab2_1
### Figure 8
$ ldd procinfo_dynamic
linux-vdso.so.1 (0x0000ff3d8005c000)
libc.so.6 => /usr/lib/aarch64-linux-gnu/libc.so.6 (0x0000ff3d7fe00000)
/lib/ld-linux-aarch64.so.1 (0x0000ff3d80020000)
$ ldd procinfo_static
not a dynamic executable

[Figure 8 done - take screenshot, then press Enter]

```

Figure 8: Runtime dependencies resolved with ldd

Task 2 Summary

Dynamic linking gave a small executable that depends on shared libraries at runtime. Static linking gave a much larger, self-contained executable that ldd treats as having no dynamic dependencies.

Task 3: ELF Executable Structure

ELF Header

ELF (Executable and Linkable Format) is the binary format used by Linux executables. The ELF header at the start of the file holds global metadata: file format, word size, byte order, file type, target machine and entry point. For `procinfo_dynamic` it showed a 64-bit, little-endian, position-independent executable (type DYN) for x86-64, with an entry point of `address`. The magic bytes `7f 45 4c 46` spell ".ELF" and mark it as a valid ELF file.

```
readelf -h procinfo_dynamic
```

```
### Figure 9
$ readelf -h procinfo_dynamic | grep -E "Magic|Class|Data|Type|Machine|Entry point"
Magic:  7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
Class:                               ELF64
Data:                               2's complement, little endian
Type:                               DYN (Position-Independent Executable file)
Machine:                             AArch64
Entry point address:                 0x900

[Figure 9 done - take screenshot, then press Enter]
```

Figure 9: ELF header of the dynamically linked executable

Program Headers

Program headers tell the OS loader which parts of the file to map into memory. The INTERP entry names the dynamic linker, which on my system is `interpreter path`. The LOAD segments show the memory layout: R for read-only data, R E for executable code, and RW for writable data. GNU_STACK was marked `RW / RWE`, meaning the stack is `readable and writable but not executable (the secure default) / executable`.

```
readelf -l procinfo_dynamic
```

```
### Figure 10
$ readelf -W -l procinfo_dynamic | grep -E "INTERP|LOAD|GNU_STACK|interpreter"
INTERP      0x0000294 0x00000000000000294 0x00000000000000294 0x000001b 0x00001b R   0x1
[Requesting program interpreter: /lib/ld-linux-aarch64.so.1]
LOAD        0x0000000 0x0000000000000000 0x0000000000000000 0x0000cc4 0x0000cc4 R E 0x10000
LOAD        0x0000140 0x00000000000001fd40 0x00000000000001fd40 0x00002d0 0x00002d8 RW 0x10000
GNU_STACK   0x0000000 0x0000000000000000 0x0000000000000000 0x0000000 0x0000000 RW 0x10

[Figure 10 done - take screenshot, then press Enter]
```

Figure 10: Program headers, interpreter path, LOAD segments and stack permissions

Knowledge Check Questions

Question 1: Difference between a C header file and a shared library

A header file is source text containing declarations, such as function prototypes, typedefs, structures and macros. The compiler reads it to check that calls are written correctly, but it holds no executable code. A shared library is a compiled binary containing the real machine code, loaded into memory at runtime. For example, `stdio.h` declares `printf`, and `libc.so.6` contains the code that runs when `printf` is called.

Question 2: Static and dynamic linking

Static linking copies the needed library code into the executable at build time, giving a larger program that runs without external `.so` files. Dynamic linking stores only references to shared libraries and lets the dynamic linker load them at runtime, giving a smaller program that shares library code in memory but fails to start if a required library is missing.

Question 3: ELF header and the commands to check headers

The ELF header is the metadata block at the very start of an ELF file. It identifies the file as ELF and records the class (32 or 64-bit), byte order, file type, target machine architecture and entry point address. The commands are:

```
readelf -h <file>    # ELF header
readelf -l <file>    # program headers
readelf -d <file>    # dynamic section
ldd <file>           # resolved shared library dependencies
```

Question 4: What does a bookmark mean? Example

A bookmark is a saved reference to a specific location so that you can return to it quickly without searching again. For example, a web browser bookmark stores the URL of this lab's GitHub repository, and in a Word document a bookmark can mark the Knowledge Check section so that a link can jump straight to it. **Adjust if your instructor meant something different by this question.**

Question 5: Documentation and GitHub submission

This documentation was pushed to GitHub at: **your repository link**

Conclusion

This lab showed how a C program uses declarations from header files and implementations from the GNU C Library. I located the headers and library files, built the process-information program, compared static and dynamic linking, and inspected the resulting ELF binary with `readelf` and `ldd`. The results showed how the system identifies the file format, maps memory segments, loads the dynamic linker and resolves the `libc` dependency. Dynamic linking keeps executables small by sharing library code at runtime, while static linking trades a larger file for independence from shared libraries.